

操作系统

实验一

Linux 命令解释程序设计与实现

实验报告

学期 2026—2027 第一学期

学院 网络空间安全学院

姓名 王旭升

学号 24281204

班级 保密 2401

编制日期 2026 年 10 月 2 日

开发方式：我采用 Agent 协助开发，在报告中说明实现原理、验证结果与功能边界。

实验目的

理解 Linux 命令解释器如何读取、解析和执行命令，区分内部命令与外部命令；通过进程创建、程序替换、等待回收、环境变量和文件描述符操作，实现可实际编译运行的 Shell，并用测试代码、运行结果和 strace 建立证据链。

本实验中，我提供指导书与要求，采用 Agent 协助编写代码、执行 Linux 测试并整理材料。我负责上传仓库和提供 GitHub 运行产物，结合源码与日志整理关键原理。Linux 测试和终端截图来自开发环境及 GitHub 运行环境。

实验任务

依据上传的《实验课题 1_Linux 命令解释程序设计与实现》（2022 年 3 月修订）及教师补充要求，实现提示符、输入解析、五条以上内部命令、exec 系列执行外部程序、至少一个内部命令直接调用系统调用接口，以及 HOME/PATH 支持。

实际完成七条内部命令：cd、pwd、echo、cat、touch、rm、exit。扩展完成 I/O 重定向 >、>>、< 与单个管道 |。通配符、多级管道、引号转义、后台作业和完整作业控制未实现，不作为已支持功能申报。

开发运行与测试环境

实际验证环境为 Ubuntu 24.04.3 LTS、Linux 6.18.44、x86_64；gcc 13.3.0、GNU Make 4.3、strace 6.8、Git 2.51.1。测试还使用 Python 3、util-linux script 和 GNU coreutils。工具已存在，无需安装 gcc、make 或 strace。

最初的受限执行视图没有 /tmp、/proc，且 ptrace 被禁止；最终复验在允许这些能力的 Linux 环境执行。两种视图的版本信息与权限差异均留有记录。我使用 Mac，但不是本项目的 Linux 测试平台，也没有通过未提供的 SSH 连接操作我的虚拟机。

编译命令为 make clean all，参数为 -std=c11 -Wall -Wextra -Wpedantic -O2 -g。最终编译成功且无 warning。运行 ./myshell 即完成启用测试，不替换系统登录 Shell。

实验步骤

第一阶段建立最小 Shell：交互提示符、getline、简单分词、exit、fork/execvp/waitpid。首次编译通过，ls、ls -l、whoami、uname -a、echo hello 等九项检查通过，PTY 中看到提示符。

第二阶段依次加入 cd、pwd、echo、cat、touch、rm。每加入一个内部命令，均重新编译、正常测试、异常测试和回归，并单独提交 Git。cd 最初因环境缺少 /tmp 测试失败，原日志保留，未伪造成功。

第三阶段验证 HOME/PATH 的继承、缺失初始化、整参数展开与自定义 PATH 搜索。累计三十三项通过。同时处理 stdio 与 read 混用的预读和刷新问题，增加 stdin 消费及输出顺序测试。

第四阶段加入输出截断、追加和输入重定向，验证真实文件内容、内外部命令、重定向失败后的恢复及 /dev/full 写入失败。累计四十八项通过。

第五阶段加入单管道，验证外部/内部命令组合、十万行数据、读端提前退出、管道中 cd/exit 的隔离语义。累计六十四项通过，未继续加入未经验证的额外功能。

最后使用真实 strace 跟踪并提取证据，加入边界、重复 fd 与 PTY 异常恢复测试。复查修复 wait4 提取器只识别 resumed 行的问题，增加四个解析回归样例，再运行 tests/verify.sh 完整复验。

关键数据结构与设计

Token 包含词文本和类型；Command 保存 argv、输入路径、输出路径和追加标志。Token 拥有动态字符串，Command 借用其地址；执行后统一 free_tokens。每命令最多 127 个 argv 项，每行最多 256 个 token、65536 字节；行长检查在 getline 之后，不声称是固定内存防护。

主流程为 main → lex → parse/run_line → run_command 或 pipeline_child。交互提示符写入 stderr，批处理输入不显示提示符。lex 只展开独立参数 \$HOME 和 \$PATH，不做通配符或字段分割。

单命令由 run_command 保存必要 fd（带 CLOEXEC），redirect 使用 open/dup2/close 改变输入输出，execute 判断 builtin 或 fork/execvp。内部命令结束后 fflush，再恢复原描述符。普通 cd 在主进程执行，因为子进程 chdir 无法改变父 Shell 的当前目录。

管道由 pipe 返回读写两端，两次 fork 分别启动左右命令，左端接 stdout，右端接 stdin。显式重定向覆盖本端管道连接。父子进程均关闭不需要的端点，父进程等待两个孩子。必须先启动两端再等待，避免管道写满而死锁；保留额外写端则会阻止 EOF。

wait_child 对 EINTR 重试，并读取退出状态。普通未知命令返回 127，其他 exec 失败返回 126；管道取右端状态，不提供 pipefail。管道内 cd/exit 只影响对应子进程。

实验结果

本次 tests/verify.sh 实际完成清理编译、完整测试、分组测试和真实 strace，返回成功。69 个 subprocess 用例与 1 项 PTY 检查通过；额外 4 个合成样例仅验证日志解析器，不算作额外 Shell 运行测试。详细结果见 logs/final_test.log。

基础 33 项、重定向 15 项、管道 16 项、健壮性 5 项。测试逐项打印输入、stdout、stderr、退出码并断言，不用预先编写的假输出替代运行。持续 100 次管道与重定向之后，父子 fd 均仅为 0、1、2。

strace 跟踪中，cd /tmp 成功；外部 /usr/bin/ls 的 execve 成功；文件创建、读取、删除及重定向出现 openat/read/write/unlink/dup2；管道出现 pipe2/clone；五个代表性子进程均被 wait4 成功回收。完整原始日志与精简片段分别保存。

报告中的证据页是日志原文摘录，标明来源文件，不是个人终端截图。初始版本的成功证据复制在 logs/archive_initial/，更早失败日志与 Git 历史也保留。所有已声明功能均在 docs/evidence.md 中对应到代码、测试和证据。

完整测试和对应关系随项目提供。下方证据页摘录最终复验日志；长行只作排版折行。

I/O 重定向与日志的区别

执行 echo hello > out.txt 时，Shell 解析 >，用 open(...O_TRUNC...) 打开目标，再用 dup2 将其接到 fd 1；命令写到 stdout 的内容真正进入 out.txt。>> 改用 O_APPEND，< 将目标接到 fd 0。这一语义由文件内容断言及 strace 双重验证。

strace -o logs/strace.log 只是指定 strace 自己的跟踪日志位置；测试脚本使用 tee 采集输出也只是测试日志。两者不能作为 Shell 已实现 I/O 重定向的证据，更不能在 README 中混称重定向功能。

讨论与问题处理

真实失败包括沙箱没有 /tmp、/proc 和禁止 ptrace。获准环境中完成了对应复验，但 /proc/PID/task/PID/children 接口仍不可用，因此改为 fd 检查与 strace 的 wait4 返回证据，不声称原检查通过。

日志提取曾把 PATH 搜索中失败的候选 execve 摘入结果，后要求成功 execve；修改时又出现正则 \$ 未启用 MULTILINE 的误报，已修正并保留失败日志。进一步复查发现 wait4 可能完整打印或分段 resumed 打印，新解析器同时接受二者，拒绝未完成行与失败返回。

代码审查中，stdin 设置无缓冲以避免 getline 提前吞掉留给 cat/外部程序的数据；stdout 及时 fflush 以避免 puts 和 write 的输出顺序反转。备份 fd 设置 CLOEXEC，避免泄漏给外部程序。这些设计均有对应行为测试。

实际未遇到编译错误，不为营造开发过程而人为制造错误。当前测试没有覆盖全部资源耗尽、OOM、恢复 dup2 失败等情况，也不据此声称程序绝无 bug。

功能边界

cd 仅支持零或一个目录参数；pwd 不接受参数。echo 用空格连接参数并换行，-n 是普通文本。cat 无参数读取 stdin，在批处理下会消费后续输入直到 EOF。touch 只创建文件并保留已有数据，不更新时间戳；rm 不递归删除目录；exit 不接受状态码。

HOME/PATH 继承启动环境，缺失时分别初始化为 / 和 /usr/bin:/bin；空字符串不替换。支持独立 \$HOME/\$PATH，不支持 \${HOME} 等复杂写法。每命令最多一个输入和一个输出重定向，重复同类重定向报错。

不支持多级管道、通配符、引号、转义、含空格字面路径、分号、逻辑连接、命令替换、脚本注释、数字 fd 重定向、历史记录、后台运行或完整信号/作业控制。2> 不当作 stderr 重定向使用；Ctrl-C 不保证保留本 Shell。

结论与提交说明

现有运行证据支持指导书基础要求和两个扩展功能（I/O 重定向、单管道）已实现并通过所列测试。可以使用本地项目或 Git 历史恢复源码，在具备 ptrace、/proc、伪终端的 Linux 中运行 bash tests/verify.sh 复现。

我重点理解进程关系与文件描述符流向，并用测试和日志验证。项目已上传 GitHub，后续复验见报告末页。已有交互文档和补充文本，但部分开发对话仍缺失；过程摘要不能替代完整会话。

证据一 编译与最终综合测试

来源：logs/final_build.log、logs/final_test.log、logs/verification_summary.log。

```
rm -f myshell
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -g src/myshell.c -o myshell
PTY TRANSCRIPT:
pwd
cd /not_exist
cat not_exist.txt
rm not_exist.txt
unknown_command
echo alive
exit
[OS-LAB1] myshell$ /workspace/scratch/876e69c55179/myshell/tests/tmp
[OS-LAB1] myshell$ cd: No such file or directory
[OS-LAB1] myshell$ not_exist.txt: No such file or directory
[OS-LAB1] myshell$ not_exist.txt: No such file or directory
[OS-LAB1] myshell$ unknown_command: No such file or directory
[OS-LAB1] myshell$ alive
[OS-LAB1] myshell$
PASS prompt returns after errors
```

```
TOTAL PASS 69 subprocess cases; PTY checked in robust group
python3 tests/test_trace_parser.py
PASS: 4 trace-parser regression samples (synthetic parser inputs, not runtime evidence)
PASS: clean build, full suite, grouped tests and real strace completed
69 项是真实命令进程测试；PTY 检查单列。4 个解析器样例是合成日志输入，不冒充系统运行次数。
```

证据二 内部命令与环境变量

来源：logs/final_test.log。异常输出保持原样。

CASE cd parent

INPUT:

cd /workspace/scratch/876e69c55179/myshell

/bin/pwd

exit

STDOUT='/workspace/scratch/876e69c55179/myshell\n'

STDERR=''

EXIT=0

PASS

CASE HOME PATH expansion

INPUT:

echo \$HOME

echo \$PATH

cd

pwd

exit

STDOUT='/workspace/scratch/876e69c55179/myshell\n/usr/bin:/bin\n/workspace/scratch/876e69c55179/myshell\n'

STDERR=''

EXIT=0

PASS

CASE cat file

INPUT:

cat cat-input.txt

exit

STDOUT='alpha\nbeta\n'

STDERR=''

EXIT=0

PASS

CASE rm error

INPUT:

rm not_exist.txt

echo alive

exit

STDOUT='alive\n'

STDERR='not_exist.txt: No such file or directory\n'

EXIT=0

PASS

证据三 I/O 重定向与管道

来源：logs/final_test.log。文件内容在测试脚本中另有断言。

```
CASE output append input
INPUT:
echo hello>out.txt
echo world>>out.txt
cat<out.txt
echo terminal
```

```
STDOUT='hello\nworld\nterminal\n'
STDERR=''
EXIT=0
PASS
```

```
CASE echo write failure
INPUT:
echo hi > /dev/full
echo alive
```

```
STDOUT='alive\n'
STDERR='stdout: No space left on device\n'
EXIT=0
PASS
```

```
CASE pipe external
INPUT:
printf hello|wc -c
```

```
STDOUT='5\n'
STDERR=''
EXIT=0
PASS
```

```
CASE large pipe no deadlock
INPUT:
seq 1 100000 | wc -l
```

```
STDOUT='100000\n'
STDERR=''
EXIT=0
PASS
```

```
CASE early reader exit
INPUT:
yes | head -n 1
```

```
STDOUT='y\n'
STDERR=''
EXIT=0
PASS
```

证据四 实际系统调用

来源：logs/strace_key.log、logs/strace_reaping.log。保留原 PID、参数和返回值；完整上下文在 logs/strace.log。

```
## cd
970  chdir("/tmp") = 0

## pwd
970  getcwd("/workspace/scratch/876e69c55179/myshell", 4096) = 40
970  getcwd("/tmp", 4096) = 5
970  getcwd("/workspace/scratch/876e69c55179/myshell/tests/tmp/trace", 4096) = 56

## touch/cat
970  openat(AT_FDCWD, "test.txt", O_WRONLY|O_CREAT, 0666) = 3
970  openat(AT_FDCWD, "test.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
970  openat(AT_FDCWD, "test.txt", O_RDONLY) = 3

## rm
970  unlink("test.txt") = 0

## external
971  execve("/usr/bin/ls", ["ls", "-l"], 0x7ffde982a288 /* 79 vars */) = 0

## redirection open
970  openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
970  openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_APPEND, 0666) = 4
970  openat(AT_FDCWD, "out.txt", O_RDONLY) = 4

## redirection dup2
970  dup2(4, 1) = 1
970  dup2(3, 1) = 1
970  dup2(4, 1) = 1

## read
970  read(0, "hello\nworld\n", 8192) = 12

## write
970  write(1, "hello\n", 6) = 6
970  write(1, "hello\nworld\n", 12) = 12
974  write(1, "hello", 5 <unfinished ...>

## pipe
970  pipe2([3, 4], 0) = 0
970  pipe2([3, 4], 0) = 0

## fork wrapper
970  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
        child_tidptr=0x7fe7a8656a10) = 971
970  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
        child_tidptr=0x7fe7a8656a10) = 972
970  clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD <unfinished
...>

## reaping
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 971
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 972
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 973
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 971
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 972
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 973
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 974
970  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 975
PASS: all 5 representative child processes reaped
```

功能与测试证据对应表

功能	代码函数	测试名称或操作	证据文件 (logs/)
提示符与异常恢复	main	PTY TRANSCRIPT	final_test.log
cd / pwd	builtin	cd parent / pwd	final_test.log; strace_cd_pwd.log
echo	builtin	echo spaces / excessive arguments	basic_test.log
cat	builtin	cat file / cat stdin / cat error	basic_test.log; strace_files.log
touch / rm	builtin	touch create/preserve; rm file/error	basic_test.log; strace_files.log
exit	builtin	exit arguments; pipe exit isolated	basic_test.log; pipe_test.log
HOME / PATH	main / lex / execute	HOME PATH expansion; custom PATH	basic_test.log; strace_exec.log
外部命令	execute	ls、whoami、uname; 成功 execve	basic_test.log; strace_exec.log
> / >> / <	redirect / run_command	output append input; truncate	redirect_test.log; strace_key.log
单管道	run_line / pipeline_child	pipe external; builtin; large pipe	pipe_test.log; strace_pipe.log
fd 关闭与回收	run_command / wait_child	100 轮 fd probe; 实际 wait4	final_test.log; strace_reaping.log
解析边界与限制	lex / parse	line/token limit; unsupported glob	final_test.log
摘录脚本修复	extract_trace.py	4 个合成回归样例	trace_parser_test.log

所有 C 函数位于 src/myshell.c; 完整命令与断言位于 tests/run_tests.py。

关键原理说明

1. 为什么 fork + exec? fork 保留一个继续解释命令的父进程，同时创建子进程。execvp 把子进程替换成外部命令。若主 Shell 自己 exec 成功，它就不再是解释器。
 2. 父子分别做什么? execute 中子进程 execvp，失败 perror 后 _exit；父进程 wait_child 等待。管道中两个子进程分别读写，父进程关闭端点并等待两者。
 3. 为什么 cd 在父进程? 当前目录是进程属性。fork 后子进程继承目录，但 chdir 只更改调用进程；不能反向更改父进程。run_command 中 builtin 在父进程执行。管道中的 cd 则按隔离语义在子进程执行，已专门测试。
 4. exec 为什么成功不返回? 内核用新可执行文件重建进程地址空间并进入新程序入口，PID 保持；原调用点不再存在。只有加载失败才返回 -1，因此 perror/_exit 放在 execvp 之后。
 5. PATH 做什么? 是可执行文件目录列表。execvp 用无斜杠的命令名逐目录尝试，最终调用 execve；strace 中先出现若干 ENOENT 再出现 /usr/bin/ls 成功，是搜索过程，不等于命令失败。
 6. 0/1/2 是什么? 约定的 stdin/stdout/stderr 文件描述符，是进程描述符表的索引，不是固定文件。提示符走 fd 2，使 fd 1 的命令输出可重定向。FILE * stdout 是 C 库缓冲对象，与内核 fd 1 不同。
 7. open + dup2 为什么实现 >? open 用 O_WRONLY|O_CREAT|O_TRUNC 创建/截断文件，dup2(fd,1) 让 fd 1 指向它。后续 write(1,...) 就进入文件。追加换成 O_APPEND。执行 builtin 前保存 fd，fflush 后恢复，避免输出在恢复后才被缓冲库写出。
 8. 管道为何两个 fd? pipe 返回读端和写端；数据从写端进入内核缓冲，从读端取出。左子进程把写端复制到 1，右子进程把读端复制到 0。
 9. 为何关闭不用的端? 只有所有写端关闭，读者才看见 EOF；多余读端会影响 SIGPIPE/EPIPE。父进程不传输数据，应关闭两端；子进程 dup2 后关闭原端点。要先启动两端再 wait，避免管道写满而无人读取。
 10. strace 如何看到调用? 它利用 Linux ptrace 跟踪机制观察调用进入、返回和参数。-f 跟踪新子进程。沙箱可能禁止 ptrace，所以安装成功不等于能跟踪；本次先失败，再在获准环境中实际成功。
 11. C 库和系统调用什么关系? puts/fputs 可先缓冲，最终经 write 写入内核；open/read/write/chdir/unlink 是系统调用的用户态 C 接口。glibc 可选择 openat/clone/pipe2/wait4 等内核入口，因此不能只按 C 函数同名搜索。至少 cat/rm/cd 明确直接使用这些接口，不是借助 system 启动外部工具。
 12. waitpid 为什么重要? 读取退出状态并回收子进程内核记录，避免僵尸。wait_child 在 EINTR 时重试，run_line 对左右子进程均等待。右端决定管道状态；左端可能因读者提前关闭收到 SIGPIPE，这是管道机制的一部分。
- 补充：stdin 设置无缓冲，避免 getline 预读吞掉原本留给 cat 或外部程序的输入；每条内部命令输出及时 fflush，避免 puts 与 write 混用时顺序反转；备份 fd 设置 CLOEXEC，防止外部程序继承它们。

附录 源程序完整清单

当前 src/myshell.c, 共 333 行; 左侧为源文件行号, 长行仅视觉折行。

```
001 #define _POSIX_C_SOURCE 200809L
002 #include <errno.h>
003 #include <fcntl.h>
004 #include <stdio.h>
005 #include <stdlib.h>
006 #include <string.h>
007 #include <sys/types.h>
008 #include <sys/wait.h>
009 #include <unistd.h>
010
011 #define MAX_ARGS 128
012
013 static int wait_child(pid_t pid)
014 {
015     int status;
016     while (waitpid(pid, &status, 0) < 0) {
017         if (errno == EINTR) continue;
018         perror("waitpid");
019         return 1;
020     }
021     return WIFEXITED(status) ? WEXITSTATUS(status) : 128 + WTERMSIG(status);
022 }
023
024 /* Return -1 for external commands; builtins run in the shell process. */
025 static int builtin(char **a, int *quit)
026 {
027     if (!strcmp(a[0], "exit")) {
028         if (a[1]) { fprintf(stderr, "exit: no arguments supported\n"); return 2; }
029         *quit = 1;
030         return 0;
031     }
032     if (!strcmp(a[0], "cd")) {
033         if (a[1] && a[2]) { fprintf(stderr, "cd: expected zero or one path\n"); return 2; }
034         const char *path = a[1] ? a[1] : getenv("HOME");
035         if (!path) { fprintf(stderr, "cd: HOME is not set\n"); return 1; }
036         /* Must execute in parent, so future commands inherit the new cwd. */
037         if (chdir(path) < 0) { perror("cd"); return 1; }
038         return 0;
039     }
040     if (!strcmp(a[0], "pwd")) {
041         if (a[1]) { fprintf(stderr, "pwd: no arguments supported\n"); return 2; }
042         char *cwd = getcwd(NULL, 0);
043         if (!cwd) { perror("pwd"); return 1; }
044         int failed = puts(cwd) == EOF;
045         free(cwd);
046         if (failed) { perror("pwd"); return 1; }
047         return 0;
048     }
049     if (!strcmp(a[0], "echo")) {
050         /* Intentionally literal: no -n option and no backslash escapes. */
051         for (size_t i = 1; a[i]; ++i) {
052             if ((i > 1 && putchar(' ') == EOF) || fputs(a[i], stdout) == EOF) {
053                 perror("echo"); return 1;
054             }
055         }
056         if (putchar('\n') == EOF) { perror("echo"); return 1; }
057         return 0;
058     }
059     if (!strcmp(a[0], "cat")) {
060         int result = 0;
061         for (size_t i = 1; i == 1 || a[i]; ++i) {
062             int fd = a[i] ? open(a[i], O_RDONLY) : STDIN_FILENO;
063             if (fd < 0) { perror(a[i]); result = 1; continue; }
064             char buffer[8192];
065             for (;;) {
066                 ssize_t n = read(fd, buffer, sizeof(buffer));
067                 if (n < 0 && errno == EINTR) continue;
068                 if (n < 0) { perror("cat: read"); result = 1; break; }

```

```

069         if (!n) break;
070         ssize_t offset = 0;
071         while (offset < n) {
072             ssize_t w = write(STDOUT_FILENO, buffer + offset, (size_t)(n - offset));
073             if (w < 0 && errno == EINTR) continue;
074             if (w <= 0) { perror("cat: write"); result = 1; break; }
075             offset += w;
076         }
077         if (offset < n) break;
078     }
079     if (a[i] && close(fd) < 0) { perror("cat: close"); result = 1; }
080     if (!a[i]) break;
081 }
082 return result;
083 }
084 if (!strcmp(a[0], "touch")) {
085     if (!a[1]) { fprintf(stderr, "touch: expected file(s)\n"); return 2; }
086     int result = 0;
087     for (size_t i = 1; a[i]; ++i) {
088         /* Create if absent; preserve existing bytes (no O_TRUNC). */
089         int fd = open(a[i], O_WRONLY | O_CREAT, 0666);
090         if (fd < 0) { perror(a[i]); result = 1; continue; }
091         if (close(fd) < 0) { perror("touch: close"); result = 1; }
092     }
093     return result;
094 }
095 if (!strcmp(a[0], "rm")) {
096     if (!a[1]) { fprintf(stderr, "rm: expected file(s)\n"); return 2; }
097     int result = 0;
098     for (size_t i = 1; a[i]; ++i) {
099         if (unlink(a[i]) < 0) { perror(a[i]); result = 1; }
100     }
101     return result;
102 }
103 return -1;
104 }
105
106 static int execute(char **a, int *quit)
107 {
108     int result = builtin(a, quit);
109     if (result >= 0) return result;
110     pid_t pid = fork();
111     if (pid < 0) { perror("fork"); return 1; }
112     if (pid == 0) {
113         execvp(a[0], a);
114         int code = errno == ENOENT ? 127 : 126;
115         perror(a[0]);
116         _exit(code); /* No duplicate flushing of inherited stdio buffers. */
117     }
118     return wait_child(pid);
119 }
120
121
122 #define MAX_TOKENS 256
123 #define MAX_LINE 65536
124
125 typedef struct {
126     char *text;
127     int kind; /* 0=word, '<', '>', 'A'=append, '|' */
128 } Token;
129
130 typedef struct {
131     char *args[MAX_ARGS];
132     const char *input, *output;
133     int append;
134 } Command;
135
136 static void free_tokens(Token *tokens, size_t n)
137 {
138     for (size_t i = 0; i < n; ++i) free(tokens[i].text);
139 }
140
141 static int lex(const char *line, Token *tokens, size_t *n)
142 {

```

```

143     *n = 0;
144     const char *p = line;
145     while (*p) {
146         if (strchr("\t\r\n", *p)) { ++p; continue; }
147         if (*n == MAX_TOKENS) { fprintf(stderr, "parse: too many tokens\n"); return 2; }
148         if (strchr("\"'\\;&", *p)) {
149             fprintf(stderr, "parse: quotes, escapes, ; and & are unsupported\n"); return 2;
150         }
151         int kind = 0;
152         const char *start = p;
153         if (strchr("<|", *p)) {
154             kind = *p++;
155             if (kind == '>' && *p == '>') { kind = 'A'; ++p; }
156         } else {
157             while (*p && !strchr("\t\r\n<>|\"'\\;&", *p)) ++p;
158         }
159         char *word = strndup(start, (size_t)(p - start));
160         if (!word) { perror("strndup"); return 1; }
161         if (!kind && (!strcmp(word, "$HOME") || !strcmp(word, "$PATH"))) {
162             const char *value = getenv(word + 1);
163             free(word);
164             word = strdup(value ? value : "");
165             if (!word) { perror("strdup"); return 1; }
166         }
167         tokens[*n] = (Token){word, kind};
168         ++*n;
169     }
170     return 0;
171 }
172
173 static int parse(Token *tokens, size_t n, Command *cmd)
174 {
175     size_t argc = 0;
176     memset(cmd, 0, sizeof(*cmd));
177     for (size_t i = 0; i < n; ++i) {
178         int kind = tokens[i].kind;
179         if (!kind) {
180             if (argc == MAX_ARGS - 1) goto syntax;
181             cmd->args[argc++] = tokens[i].text;
182         } else {
183             if (kind == '|' || i + 1 == n || tokens[i + 1].kind) goto syntax;
184             const char *path = tokens[++i].text;
185             if (kind == '<') {
186                 if (cmd->input) goto syntax;
187                 cmd->input = path;
188             } else {
189                 if (cmd->output) goto syntax;
190                 cmd->output = path;
191                 cmd->append = kind == 'A';
192             }
193         }
194     }
195     if (!argc) goto syntax;
196     return 0;
197 syntax:
198     fprintf(stderr, "parse: invalid syntax, duplicate redirection or too many arguments\n");
199     return 2;
200 }
201
202 static int redirect(const Command *cmd)
203 {
204     for (int target = 0; target <= 1; ++target) {
205         const char *path = target ? cmd->output : cmd->input;
206         if (!path) continue;
207         int flags = target ? O_WRONLY | O_CREAT | (cmd->append ? O_APPEND : O_TRUNC) :
O_RDONLY;
208         int fd = open(path, flags, 0666);
209         if (fd < 0) { perror(path); return 1; }
210         if (fd != target) {
211             if (dup2(fd, target) < 0) { perror("dup2"); close(fd); return 1; }
212             if (close(fd) < 0) { perror("close"); return 1; }
213         }
214     }
215     return 0;

```

```

216 }
217
218 static int run_command(const Command *cmd, int *quit)
219 {
220     int saved[2] = {-1, -1}, result = 1;
221     /* Save only changed fds. CLOEXEC prevents leaking backup fds to external programs. */
222     for (int i = 0; i < 2; ++i) {
223         if (!(i ? cmd->output : cmd->input)) continue;
224         saved[i] = fcntl(i, F_DUPFD_CLOEXEC, 3);
225         if (saved[i] < 0) { perror("fcntl: save fd"); goto restore; }
226     }
227     if (redirect(cmd)) goto restore;
228     result = execute((char **)cmd->args, quit);
229     if (fflush(stdout) == EOF) { perror("stdout"); clearerr(stdout); result = 1; }
230 restore:
231     for (int i = 0; i < 2; ++i) {
232         if (saved[i] < 0) continue;
233         if (dup2(saved[i], i) < 0) { perror("dup2: restore"); result = 1; *quit = 1; }
234         if (close(saved[i]) < 0) { perror("close: saved fd"); result = 1; }
235     }
236     clearerr(stdin);
237     return result;
238 }
239
240 /* Pipeline builtins run in children: cd/exit affect only their pipeline side. */
241 static void pipeline_child(const Command *cmd, int read_end, int write_end, int side)
242 {
243     int fd = side == 0 ? write_end : read_end;
244     int target = side == 0 ? STDOUT_FILENO : STDIN_FILENO;
245     if (dup2(fd, target) < 0) { perror("dup2: pipe"); _exit(1); }
246     if (close(read_end) < 0 || close(write_end) < 0) { perror("close: pipe"); _exit(1); }
247     /* Explicit redirection overrides this side's pipe connection. */
248     if (redirect(cmd)) _exit(1);
249     int quit = 0;
250     int result = builtin((char **)cmd->args, &quit);
251     if (result >= 0) {
252         if (fflush(stdout) == EOF) { perror("stdout"); result = 1; }
253         _exit(result);
254     }
255     execvp(cmd->args[0], cmd->args);
256     int code = errno == ENOENT ? 127 : 126;
257     perror(cmd->args[0]);
258     _exit(code);
259 }
260
261 static int run_line(Token *tokens, size_t n, int *quit)
262 {
263     size_t split = n;
264     for (size_t i = 0; i < n; ++i) {
265         if (tokens[i].kind != '|') continue;
266         if (split != n) { fprintf(stderr, "parse: only one pipe supported\n"); return 2; }
267         split = i;
268     }
269     Command left, right;
270     int result = parse(tokens, split, &left);
271     if (result) return result;
272     if (split == n) return run_command(&left, quit);
273     result = parse(tokens + split + 1, n - split - 1, &right);
274     if (result) return result;
275     int fds[2];
276     if (pipe(fds) < 0) { perror("pipe"); return 1; }
277     pid_t first = fork();
278     if (first == 0) pipeline_child(&left, fds[0], fds[1], 0);
279     if (first < 0) { perror("fork"); close(fds[0]); close(fds[1]); return 1; }
280     pid_t second = fork();
281     if (second == 0) pipeline_child(&right, fds[0], fds[1], 1);
282     /* Parent owns neither endpoint. Keeping a writer open prevents EOF. */
283     int close_error = 0;
284     if (close(fds[0]) < 0) { perror("close: pipe read"); close_error = 1; }
285     if (close(fds[1]) < 0) { perror("close: pipe write"); close_error = 1; }
286     if (second < 0) {
287         perror("fork"); wait_child(first); return 1;
288     }
289     /* Both children must exist before waiting, otherwise large output can deadlock. */

```

```

290     wait_child(first);
291     result = wait_child(second);
292     return close_error ? 1 : result;
293 }
294
295 int main(void)
296 {
297     /* Inherit caller environment; use documented defaults only if absent. */
298     if ((!getenv("HOME") && setenv("HOME", "/", 1) < 0) ||
299         (!getenv("PATH") && setenv("PATH", "/usr/bin:/bin", 1) < 0)) {
300         perror("setenv"); return 1;
301     }
302     /* No stdio read-ahead: cat/external commands may consume the same fd 0. */
303     if (setvbuf(stdin, NULL, _IONBF, 0) != 0) {
304         fprintf(stderr, "setvbuf failed\n"); return 1;
305     }
306     char *line = NULL;
307     size_t capacity = 0;
308     int quit = 0, result = 0;
309     while (!quit) {
310         if (isatty(STDIN_FILENO)) {
311             fputs("[OS-LAB1] myshell$ ", stderr);
312             fflush(stderr);
313         }
314         if (getline(&line, &capacity, stdin) < 0) {
315             if (ferror(stdin)) { perror("getline"); result = 1; }
316             break;
317         }
318         if (strlen(line) > MAX_LINE) {
319             fprintf(stderr, "parse: line too long\n"); result = 2; continue;
320         }
321         Token tokens[MAX_TOKENS];
322         size_t n = 0;
323         result = lex(line, tokens, &n);
324         if (!result && n) {
325             result = run_line(tokens, n, &quit);
326         }
327         free_tokens(tokens, n);
328         /* Commit builtin output before a later fork or descriptor change. */
329         if (fflush(stdout) == EOF) { perror("stdout"); result = 1; clearerr(stdout); }
330     }
331     free(line);
332     return result;
333 }

```

附录 Makefile 与复验入口

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -Wpedantic -O2 -g
```

```
.PHONY: all clean test
all: myshell
myshell: src/myshell.c
    $(CC) $(CFLAGS) $< -o $@
clean:
    rm -f myshell
test: myshell
    python3 tests/run_tests.py all
    python3 tests/test_trace_parser.py
```

实际 Makefile 配方使用 Tab。复验入口：bash tests/verify.sh。

```
#!/bin/bash
# One reproducible entry point for the teacher's Linux host.
set -euo pipefail
cd "$(dirname "$0")/.."
mkdir -p logs
{
    uname -a
    cat /etc/os-release
    gcc --version
    make --version
    strace --version
    git --version
} > logs/verification_environment.log 2>&1
make clean all > logs/final_build.log 2>&1
make test > logs/final_test.log 2>&1
bash tests/test_basic.sh > logs/basic_runner.log 2>&1
bash tests/test_redirect.sh > logs/redirect_runner.log 2>&1
bash tests/test_pipe.sh > logs/pipe_runner.log 2>&1
bash tests/trace.sh > logs/strace_extract_output.log 2>&1
sha256sum src/myshell.c Makefile myshell > logs/artifact_hashes.log
printf 'PASS: clean build, full suite, grouped tests and real strace completed\n' | tee
    logs/verification_summary.log
```

当前编译对象校验值

8a53757850b66ce3c1dfc665808fceb4a0d829f8da61cb7236a8dc838b0ab194	src/myshell.c
c25e85082440917832ee02aa76ab55b24c2f13056ba55400e106d3f3dce268f5	Makefile
12e37c70bd3480a31739aa43ebb3d1b98b7d9a7b1ebb02efac2ac54c32e23c77	myshell

我已上传 GitHub 仓库并保存后续运行产物，具体结果见报告末页。交互材料已附 Word 与补充 TXT，但部分开发对话仍有缺失。

补充 实际终端截图

2026 年 10 月 7 日 UTC 补充。以下为 Agent 在 Linux 中运行真实 PTY 与 xterm 后截取的窗口，含姓名拼音与学号。图 10、14 明确查看历史记录；图 12、13 保留本次环境限制，未声称本次完整复验成功。

图 1 当前 Linux 环境与工具版本

```
OS LAB 1 | Wang Xusheng | 24281204 | 01-environment
CURRENT RUN
2026-10-07T01:08:05Z
$ uname -a; cat /etc/os-release; gcc --version | head -1; make --version | head -1; strace --version | head -1; git --ve
rsion
Linux 7745960babb0 6.18.44 #1 SMP Sat Sep 26 20:02:31 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
PRETTY_NAME="Ubuntu 24.04.3 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.3 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
GNU Make 4.3
strace -- version 6.8
git version 2.51.1

[actual command status: 0]
█
```

图 2 清理后重新编译成功

```
OS LAB 1 | Wang Xusheng | 24281204 | 02-build
CURRENT RUN
2026-10-07T01:08:06Z
$ make clean all
rm -f myshell
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -g src/myshell.c -o myshell

[actual command status: 0]
█
```

实际终端截图 续

图 3 Shell 启动及真实 PTY 提示符

```
OS LAB 1 | Wang Xusheng | 24281204 | 03-prompt
CURRENT RUN
2026-10-07T01:08:06Z
$ printf %s 'echo OS-LAB1-started
exit
' | script -q -e -c ./myshell /dev/null
echo OS-LAB1-started
exit
[OS-LAB1] myshell$ OS-LAB1-started
[OS-LAB1] myshell$
[actual command status: 0]
```

图 4 pwd、echo 与 HOME 展开

```
OS LAB 1 | Wang Xusheng | 24281204 | 04-builtins
CURRENT RUN
2026-10-07T01:08:07Z
$ printf %s 'pwd
echo hello
echo $HOME
exit
' | script -q -e -c ./myshell /dev/null
pwd
echo hello
echo $HOME
exit
[OS-LAB1] myshell$ /workspace/scratch/876e69c55179/myshell
[OS-LAB1] myshell$ hello
[OS-LAB1] myshell$ /root
[OS-LAB1] myshell$
[actual command status: 0]
```

实际终端截图 续

图 5 外部命令运行

```
OS LAB 1 | Wang Xusheng | 24281204 | 05-external
CURRENT RUN
2026-10-07T01:08:08Z
$ printf %s 'ls -l src
whoami
uname -a
exit
' | script -q -e -c ./myshell /dev/null
ls -l src
whoami
uname -a
exit
[OS-LAB1] myshell$ total 12
-rw-r--r-- 1 root root 11532 Oct  3 06:50 myshell.c
[OS-LAB1] myshell$ root
[OS-LAB1] myshell$ Linux 7745960babd0 6.18.44 #1 SMP Sat Sep 26 20:02:31 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
[OS-LAB1] myshell$
[actual command status: 0]
```

图 6 主进程目录改变与错误后继续

```
OS LAB 1 | Wang Xusheng | 24281204 | 06-cd
CURRENT RUN
2026-10-07T01:08:08Z
$ printf %s 'pwd
cd /workspace/scratch/876e69c55179/myshell/tests/tmp
pwd
cd /not_exist
pwd
exit
' | script -q -e -c ./myshell /dev/null
pwd
cd /workspace/scratch/876e69c55179/myshell/tests/tmp
pwd
cd /not_exist
pwd
exit
[OS-LAB1] myshell$ /workspace/scratch/876e69c55179/myshell
[OS-LAB1] myshell$ [OS-LAB1] myshell$ /workspace/scratch/876e69c55179/myshell/tests/tmp
[OS-LAB1] myshell$ cd: No such file or directory
[OS-LAB1] myshell$ /workspace/scratch/876e69c55179/myshell/tests/tmp
[OS-LAB1] myshell$
[actual command status: 0]
```

实际终端截图 续

图 7 文件创建、读取、删除及异常恢复

```
OS LAB 1 | Wang Xusheng | 24281204 | 07-files
CURRENT RUN
2026-10-07T01:08:09Z
$ printf %s 'touch tests/tmp/screenshot-demo.txt
echo data > tests/tmp/screenshot-demo.txt
cat tests/tmp/screenshot-demo.txt
rm tests/tmp/screenshot-demo.txt
cat tests/tmp/screenshot-demo.txt
echo alive
exit
' | script -q -e -c ./myshell /dev/null
touch tests/tmp/screenshot-demo.txt
echo data > tests/tmp/screenshot-demo.txt
cat tests/tmp/screenshot-demo.txt
rm tests/tmp/screenshot-demo.txt
cat tests/tmp/screenshot-demo.txt
echo alive
exit
[OS-LAB1] myshell$ [OS-LAB1] myshell$ [OS-LAB1] myshell$ data
[OS-LAB1] myshell$ [OS-LAB1] myshell$ tests/tmp/screenshot-demo.txt: No such file or directory
[OS-LAB1] myshell$ alive
[OS-LAB1] myshell$
[actual command status: 0]
```

图 8 实际输出、追加和输入重定向

```
OS LAB 1 | Wang Xusheng | 24281204 | 08-redirectation
CURRENT RUN
2026-10-07T01:08:10Z
$ printf %s 'echo hello > tests/tmp/screenshot-out.txt
echo world >> tests/tmp/screenshot-out.txt
cat < tests/tmp/screenshot-out.txt
cat tests/tmp/screenshot-out.txt
exit
' | script -q -e -c ./myshell /dev/null
echo hello > tests/tmp/screenshot-out.txt
echo world >> tests/tmp/screenshot-out.txt
cat < tests/tmp/screenshot-out.txt
cat tests/tmp/screenshot-out.txt
exit
[OS-LAB1] myshell$ [OS-LAB1] myshell$ [OS-LAB1] myshell$ hello
world
[OS-LAB1] myshell$ hello
world
[OS-LAB1] myshell$
[actual command status: 0]
```

实际终端截图 续

图 9 单级管道，printf hello 计数为 5

```
OS LAB 1 | Wang Xusheng | 24281204 | 09-pipe
CURRENT RUN
2026-10-07T01:08:10Z
$ printf %s 'ls | wc -l
printf hello | wc -c
cat tests/tmp/screenshot-out.txt | wc -w
exit
' | script -q -e -c ./myshell /dev/null
ls | wc -l
printf hello | wc -c
cat tests/tmp/screenshot-out.txt | wc -w
exit
[OS-LAB1] myshell$ 8
[OS-LAB1] myshell$ 5
[OS-LAB1] myshell$ 2
[OS-LAB1] myshell$
[actual command status: 0]
```

图 10 查看历史成功 strace 原始日志（非本次新跟踪）

```
OS LAB 1 | Wang Xusheng | 24281204 | 10-strace-history
HISTORICAL TRACE FILE VIEW - NOT A NEW TRACE
2026-10-07T01:08:11Z
$ grep -E 'chdir(|execve(|"/usr/bin/ls"|out.txt.*0_|pipe2(|(' logs/strace.log; cat logs/strace_reaping.log
970 chdir("/tmp") = 0
970 chdir("/workspace/scratch/876e69c55179/myshell/tests/tmp/trace") = 0
971 execve("/usr/bin/ls", ["ls", "-l"], 0x7ffde982a288 /* 79 vars */) = 0
971 statx(AT_FDCWD, "out.txt", AT_STATX_SYNC_AS_STAT|AT_SYMLINK_NOFOLLOW|AT_NO_AUTOMOUNT, STATX_MODE|STATX_NLINK|STATX
_UID|STATX_GID|STATX_MTIME|STATX_SIZE, {stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0, stx_mode=S_IFREG|0644
, stx_size=12, ...}) = 0
970 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
970 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_APPEND, 0666) = 4
970 openat(AT_FDCWD, "out.txt", O_RDONLY) = 4
970 pipe2([3, 4], 0) = 0
972 execve("/usr/bin/ls", ["ls"], 0x7ffde982a288 /* 79 vars */) <unfinished ...>
970 pipe2([3, 4], 0) = 0
970 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 971
970 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 972
970 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 973
970 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 974
970 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 975
PASS: all 5 representative child processes reaped
```

```
[actual command status: 0]
```

实际终端截图 续

图 11 截图时的真实 Git 开发历史

```
OS LAB 1 | Wang Xusheng | 24281204 | 11-git
CURRENT HISTORY VIEW
2026-10-07T01:08:11Z
$ git log --oneline --reverse
8d7e20f init project and record environment and assignment
7d5516a implement and test minimal fork execvp waitpid shell
f44d06b add cd and adapt directory test to actual sandbox filesystem
65e5a2e add builtin pwd with normal error and regression tests
9e4780f add builtin echo with normal error and regression tests
fee772d add builtin cat with normal error and regression tests
cf4e4c9 add builtin touch with normal error and regression tests
02f67b9 add builtin rm with normal error and regression tests
025baf8 support HOME PATH initialization expansion and verify PATH lookup
bbc45e4 implement real input output append redirection with fd restoration
ecfdc51 add single pipeline with concurrent children and fd closure tests
5d30ebc add final tests real strace evidence and documented failures
3c0223d document full acceptance and add verified Chinese report with source appendix
ed7b4ca fix strace reaping parser for complete and resumed syscall lines
7679a7b revalidate all features and prepare honest agent-assisted submission
5cdc979 finalize agent-assisted report with actual evidence and source listing
71287a5 audit requirements, correct screenshot obligation and add semantic tests
bf3a963 prepare unexecuted Linux terminal screenshot workflow for user repository
0ee5266 add step-by-step Mac browser upload and screenshot run guide

[actual command status: 0]
```

图 12 本次测试：68 项后受/proc 限制，追加 18 项通过

```
OS LAB 1 | Wang Xusheng | 24281204 | 12-tests
CURRENT RUN - CHECK EXIT STATUS
2026-10-07T01:08:12Z
$ make test > logs/screenshots_local/current-suite.log 2>&1; rc=$?; grep -c "^PASS$" logs/screenshots_local/current-suite.log; tail -9 logs/screenshots_local/current-suite.log; printf "make_test_exit=%s\n" "$rc"; python3 tests/test_audit.py > logs/screenshots_local/additional.log 2>&1; ar=$?; tail -2 logs/screenshots_local/additional.log; python3 tests/test_trace_parser.py; test "$rc" = 0 && test "$ar" = 0
68
    for fn in functions: fn()
        ^^^
File "/workspace/scratch/876e69c55179/myshell/tests/run_tests.py", line 127, in test_robust
    case('parent and child fd leak after 100 pipelines', script, 'FD_OK\n')
File "/workspace/scratch/876e69c55179/myshell/tests/run_tests.py", line 20, in case
    assert p.returncode == code, (name, 'exit', code)
    ^^^^^^^^^^^^^^^^^^^^^
AssertionError: ('parent and child fd leak after 100 pipelines', 'exit', 0)
make: *** [Makefile:11: test] Error 1
make_test_exit=2
[OS-LAB1] myshell$
TOTAL PASS 18 additional subprocess cases + PTY check
PASS: 4 trace-parser regression samples (synthetic parser inputs, not runtime evidence)

[actual command status: 1]
```

实际终端截图 续

图 13 本次 strace 权限拒绝，保留真实失败

```
OS LAB 1 | Wang Xusheng | 24281204 | 13-strace-current
CURRENT RUN - PERMISSION CHECK
2026-10-07T01:08:13Z
$ strace -f -o logs/screenshots_local/strace-probe.log /bin/true
strace: test_ptrace_get_syscall_info: PTRACE_TRACEME: Operation not permitted
strace: ptrace(PTRACE_TRACEME, ...): Operation not permitted
strace: PTRACE_SETOPTIONS: Operation not permitted
strace: cleanup: waitpid(-1, __WALL): No child processes

[actual command status: 1]
```

图 14 查看历史完整通过日志并核对当前编译对象哈希

```
OS LAB 1 | Wang Xusheng | 24281204 | 14-test-history
HISTORICAL FULL PASS FILE VIEW - NOT A NEW TEST
2026-10-07T01:08:13Z
$ tail -7 logs/final_test.log; cat logs/verification_summary.log; sha256sum -c logs/artifact_hashes.log
[OS-LAB1] myshell$ alive
[OS-LAB1] myshell$
PASS prompt returns after errors

TOTAL PASS 69 subprocess cases; PTY checked in robust group
python3 tests/test_trace_parser.py
PASS: 4 trace-parser regression samples (synthetic parser inputs, not runtime evidence)
PASS: clean build, full suite, grouped tests and real strace completed
src/myshell.c: OK
Makefile: OK
myshell: OK

[actual command status: 0]
```

理解与体会

我对 Shell 的理解是：它负责解析命令、组织进程和连接文件描述符。输入一条命令后，需要先判断它是内部命令还是外部程序，再决定怎样执行。我认为把解析、执行和文件描述符处理分开，能更清楚地定位问题，也能把代码与操作系统原理对应起来。

我用 `cd` 来理解父子进程的区别。当前工作目录属于进程，子进程调用 `chdir` 不能修改父 Shell 的目录，所以普通 `cd` 必须在主进程执行。外部命令则先 `fork`，再由子进程 `execvp` 加载程序；`exec` 成功后替换程序映像，不会回到原来的调用点。父进程继续作为 Shell 存在，并用 `waitpid` 等待和回收子进程。管道里的 `cd` 只影响子进程，这与普通 `cd` 的执行位置不同。

我理解的重定向，是改变命令实际使用的输入或输出通道。0、1、2 分别是标准输入、标准输出和标准错误。`echo hello > out.txt` 通过 `open` 打开文件，再用 `dup2` 让描述符 1 指向它，后续输出才会写入文件。主进程执行内部命令后，还需要刷新缓冲并恢复原描述符。我特别区分了这与 `strace -o` 的用途：后者保存跟踪日志，不能证明 Shell 实现了 I/O 重定向。

管道让我关注到并发和文件描述符的生命周期。`pipe` 提供读端和写端，分别连接右命令的输入与左命令的输出。若先等待左端结束才启动右端，大量输出可能填满缓冲区而阻塞。无用端点也必须关闭，否则读端可能等不到 EOF。因此我认为，项目中的十万行数据和读端提前退出测试，比只看一条短管道的结果更有说服力。

对于环境变量，我理解 `PATH` 决定 `execvp` 如何搜索不带斜杠的外部命令；`HOME` 用于无参 `cd`，以及本项目支持的独立参数展开。分析 `strace` 时，我会结合 PID、参数和返回值判断行为。C 库接口与内核调用不一定同名，例如 `open` 可能对应 `openat`，`fork` 对应 `clone`，`waitpid` 对应 `wait4`，不能因为名称不同就判断功能没有执行。

我认为验收应同时检查输出、退出码、文件内容和系统调用记录。开发阶段受限环境中的失败与后续 GitHub 成功复验属于不同运行，应分别保留。我采用 Agent 协助开发，并通过代码、测试与日志梳理这些原理。报告中的功能范围以实际证据为准；引号、通配符、多级管道和完整作业控制尚未实现，不能当作已有功能申报。

GitHub 完整复验与材料更新

我将 GitHub 运行产物保存在 os1.zip 内的 linux-terminal-evidence.zip 中，作为本次更新依据。该次截图记录时间为 2026 年 10 月 7 日 04:59 UTC。前文保留开发阶段及受限环境的历史结果，本页记录后续成功复验，不将历史失败改写为成功。

本次运行环境为 Ubuntu 24.04.5 LTS、Linux 6.17.0-1022-azure、x86_64；gcc 13.3.0、Make 4.3、strace 6.8、Git 2.55.0，与前述开发环境分别记录。

核验产物中的 12 个截图阶段退出码均为 0，清理编译成功；69 项命令测试与 PTY 检查、18 项追加测试与 PTY 检查、4 项解析回归通过。真实 strace 中，五个创建的子进程均有对应 wait4 回收记录。完整新证据位于 evidence/github-20261007/。

我的项目仓库为 <https://github.com/wwwfirstsleep/os-2026-lab>。核对时，仓库中的 Shell 源码、Makefile 及两份主要测试脚本与开发版本一致。本次仅修订文档叙述，已有测试证据对应的代码未作修改。

我已保存交互记录 Word 和补充 TXT，但中间开发对话仍有缺失，尚未形成完整会话记录。已有材料能说明部分交流过程，缺失的记录仍需要补齐。

图 15 GitHub 完整复验成功结果

